

Diffusion Models

1 Goal of Diffusion Models

Diffusion models are probabilistic generative models. Their goal is to learn a distribution close to the data distribution and then generate new samples from it.

We observe data

$$x_0 \sim p_{\text{data}}(x_0),$$

and we want to learn a model that can generate new samples resembling x_0 .

The central idea is to define two processes:

- a **forward noising process**, which gradually transforms data x_0 into Gaussian noise x_T ;
- a **reverse denoising process**, which learns how to go from noise back to data.

The forward process is fixed. The reverse process is learned.

2 Review: Gaussian Vectors and Affine Transformations

We first recall Gaussian vectors and affine Gaussian transformations. This review is essential because the whole DDPM derivation relies on manipulating Gaussian conditional distributions.

2.1 Affine Transformation of a Gaussian Vector

Let

$$X \sim \mathcal{N}(m, P),$$

where $m \in \mathbb{R}^d$ and $P \in \mathbb{R}^{d \times d}$ is a covariance matrix. Let

$$Y = AX + b + \xi,$$

where A is a matrix, b is a vector, and

$$\xi \sim \mathcal{N}(0, \Sigma)$$

is independent of X .

Then Y is Gaussian and

$$Y \sim \mathcal{N}(Am + b, APA^\top + \Sigma).$$

Indeed,

$$\begin{aligned}\mathbb{E}[Y] &= \mathbb{E}[AX + b + \xi] \\ &= A\mathbb{E}[X] + b + \mathbb{E}[\xi] \\ &= Am + b,\end{aligned}$$

and

$$\begin{aligned}\text{Cov}(Y) &= \text{Cov}(AX + \xi) \\ &= A\text{Cov}(X)A^\top + \text{Cov}(\xi) \\ &= APA^\top + \Sigma.\end{aligned}$$

This result is used repeatedly in diffusion models because each noising step is an affine Gaussian transformation.

2.2 Joint Gaussian Structure

If

$$X \sim \mathcal{N}(m, P),$$

and

$$Y \mid X = x \sim \mathcal{N}(Ax + b, \Sigma),$$

then the joint vector (X, Y) is Gaussian:

$$\begin{pmatrix} X \\ Y \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} m \\ Am + b \end{pmatrix}, \begin{pmatrix} P & PA^\top \\ AP & APA^\top + \Sigma \end{pmatrix}\right).$$

The covariance blocks are obtained as follows:

$$\begin{aligned}\text{Cov}(X, Y) &= \text{Cov}(X, AX + \xi + b) \\ &= \text{Cov}(X, X)A^\top \\ &= PA^\top, \\ \text{Cov}(Y, X) &= AP.\end{aligned}$$

2.3 Conditional Gaussian Formula

For a joint Gaussian vector

$$\begin{pmatrix} X \\ Y \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} m_X \\ m_Y \end{pmatrix}, \begin{pmatrix} \Sigma_{XX} & \Sigma_{XY} \\ \Sigma_{YX} & \Sigma_{YY} \end{pmatrix}\right),$$

the conditional distribution of X given $Y = y$ is Gaussian:

$$X \mid Y = y \sim \mathcal{N}\left(m_X + \Sigma_{XY}\Sigma_{YY}^{-1}(y - m_Y), \Sigma_{XX} - \Sigma_{XY}\Sigma_{YY}^{-1}\Sigma_{YX}\right).$$

Applying this to the affine model $Y = AX + b + \xi$ gives

$$X \mid Y = y \sim \mathcal{N}\left(m + (AP)^\top(APA^\top + \Sigma)^{-1}(y - Am - b), P - (AP)^\top(APA^\top + \Sigma)^{-1}(AP)\right).$$

This formula is one of the key tools used to derive the reverse Gaussian transitions in diffusion models.

3 Forward Diffusion Process

Let x_0 denote a clean data sample. Diffusion models define a Markov chain

$$x_0 \rightarrow x_1 \rightarrow \cdots \rightarrow x_T,$$

where noise is progressively added.

A standard forward transition is

$$q(x_t \mid x_{t-1}) = \mathcal{N}(x_t; \sqrt{\alpha_t}x_{t-1}, \beta_t I),$$

where

$$\alpha_t = 1 - \beta_t,$$

and $0 < \beta_t < 1$ is a variance schedule.

Equivalently,

$$x_t = \sqrt{\alpha_t}x_{t-1} + \sqrt{\beta_t}z_t, \quad z_t \sim \mathcal{N}(0, I).$$

The joint forward law is

$$q(x_{0:T}) = p_{\text{data}}(x_0) \prod_{t=1}^T q(x_t \mid x_{t-1}).$$

The full path distribution is obtained by multiplying the initial data law by the forward Markov transitions.

4 Closed Form of $q(x_t \mid x_0)$

A crucial property of the forward process is that one can sample x_t directly from x_0 without simulating all intermediate steps.

Define

$$\bar{\alpha}_t = \prod_{s=1}^t \alpha_s.$$

Then

$$q(x_t \mid x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I).$$

Equivalently,

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}z, \quad z \sim \mathcal{N}(0, I).$$

4.1 Derivation

For $t = 1$,

$$x_1 = \sqrt{\alpha_1}x_0 + \sqrt{\beta_1}z_1.$$

Thus

$$x_1 \mid x_0 \sim \mathcal{N}(\sqrt{\alpha_1}x_0, \beta_1 I).$$

Since $\beta_1 = 1 - \alpha_1$, this matches

$$q(x_1 \mid x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_1}x_0, (1 - \bar{\alpha}_1)I).$$

Assume

$$x_{t-1} \mid x_0 \sim \mathcal{N}(\sqrt{\bar{\alpha}_{t-1}}x_0, (1 - \bar{\alpha}_{t-1})I).$$

Then

$$x_t = \sqrt{\alpha_t}x_{t-1} + \sqrt{\beta_t}z_t.$$

Using the affine Gaussian transformation result,

$$\begin{aligned} \mathbb{E}[x_t \mid x_0] &= \sqrt{\alpha_t}\sqrt{\bar{\alpha}_{t-1}}x_0 \\ &= \sqrt{\bar{\alpha}_t}x_0, \end{aligned}$$

and

$$\begin{aligned} \text{Var}(x_t \mid x_0) &= \alpha_t(1 - \bar{\alpha}_{t-1})I + \beta_t I \\ &= (\alpha_t - \alpha_t\bar{\alpha}_{t-1} + \beta_t)I \\ &= (\alpha_t + \beta_t - \bar{\alpha}_t)I \\ &= (1 - \bar{\alpha}_t)I. \end{aligned}$$

Therefore,

$$q(x_t \mid x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I).$$

4.2 Equivalent Notation

It is common to write

$$q(x_t \mid x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, \sigma_t^2 I),$$

where

$$\sigma_t = \sqrt{1 - \bar{\alpha}_t}.$$

Thus,

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sigma_t z, \quad z \sim \mathcal{N}(0, I).$$

As t becomes large, $\bar{\alpha}_t$ tends to 0 and σ_t^2 tends to 1, so

$$x_t \approx \mathcal{N}(0, I).$$

This is why the terminal variable x_T can be treated as nearly Gaussian noise.

5 Reverse Process and the Main Difficulty

The forward process is known. To generate data, we need to reverse it:

$$x_T \rightarrow x_{T-1} \rightarrow \cdots \rightarrow x_0.$$

The exact reverse joint law can be written as

$$p(x_{0:T}) = p(x_T) \prod_{s=0}^{T-1} p(x_s \mid x_{s+1}).$$

Using Bayes' rule,

$$p(x_s \mid x_{s+1}) \propto p(x_s)p(x_{s+1} \mid x_s).$$

The problem is that $p(x_s)$ is generally unknown. Thus, the exact reverse transitions are not directly computable.

Core idea. We introduce a variational reverse model

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^T p_\theta(x_{t-1} \mid x_t),$$

and learn the reverse transitions.

6 The Variational Reverse Model

A standard choice is

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t)).$$

The model learns the mean and sometimes the covariance. In many DDPM formulations, the covariance is fixed or simplified:

$$\Sigma_\theta(x_t, t) = \beta_t I$$

or another predetermined scalar multiple of the identity.

The goal is to choose θ so that the distribution generated by the reverse chain matches the data distribution.

7 Variational Objective

We want to maximize

$$\log p_\theta(x_0).$$

Since this is difficult, we use a variational bound. A useful inequality is

$$-\int \log \frac{q_\theta(x_0)}{p(x_0)} p(x_0) dx_0 \leq -\int \log \frac{q_\theta(x_{0:T})}{p(x_{0:T})} p(x_{0:T}) dx_{0:T}.$$

The essential idea is that comparing full paths gives an upper bound on comparing marginals.

In DDPM notation, the negative ELBO can be decomposed into several KL terms:

$$L(\theta) = \mathbb{E}_q \left[D_{\text{KL}}(q(x_T \mid x_0) \parallel p(x_T)) + \sum_{t=2}^T D_{\text{KL}}(q(x_{t-1} \mid x_t, x_0) \parallel p_\theta(x_{t-1} \mid x_t)) - \log p_\theta(x_0 \mid x_1) \right].$$

The most important terms are the intermediate KL divergences:

$$D_{\text{KL}}(q(x_{t-1} \mid x_t, x_0) \parallel p_\theta(x_{t-1} \mid x_t)).$$

Thus, learning the reverse model reduces to approximating the true posterior reverse transitions

$$q(x_{t-1} \mid x_t, x_0).$$

8 Posterior Reverse Transition $q(x_{t-1} \mid x_t, x_0)$

The exact distribution

$$q(x_{t-1} \mid x_t, x_0)$$

is Gaussian because all involved variables are jointly Gaussian conditional on x_0 .

We have

$$q(x_t \mid x_{t-1}) = \mathcal{N}(\sqrt{\alpha_t}x_{t-1}, \beta_t I),$$

and

$$q(x_{t-1} \mid x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_{t-1}}x_0, (1 - \bar{\alpha}_{t-1})I).$$

Using Gaussian conditioning, one obtains

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I),$$

where

$$\tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t,$$

and

$$\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t}x_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}x_t.$$

Equivalently, this can be written as

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}(x_{t-1}; \tilde{y}_t(x_t, x_0), \tilde{\sigma}_t^2 I).$$

9 Approximating the Reverse Mean

The reverse model is chosen as

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(x_{t-1}; y_\theta(x_t, t), \Sigma_{\theta,t}(x_t)).$$

The learned mean $y_\theta(x_t, t)$ should approximate the true posterior mean

$$\tilde{y}_t(x_t, x_0).$$

But during generation we do not know x_0 . Therefore, the network must infer information about x_0 from the noisy variable x_t .

From the forward formula

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sigma_t z,$$

we can solve formally for x_0 :

$$x_0 = \frac{x_t - \sigma_t z}{\sqrt{\bar{\alpha}_t}}.$$

Therefore, estimating x_0 is equivalent to estimating the noise z .

This motivates the parametrization

$$y_\theta(x_t, t) = \tilde{y}_t \left(x_t, \frac{x_t - \sigma_t \varepsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}} \right),$$

where $\varepsilon_\theta(x_t, t)$ is a neural network.

Thus, instead of directly predicting the reverse mean, the model predicts the noise used to corrupt the data.

10 Noise Prediction Objective

The standard DDPM simplification leads to the objective

$$L_{\text{simple}}(\theta) = \mathbb{E}_{t,x_0,z} \left[\|z - \varepsilon_{\theta}(x_t, t)\|^2 \right],$$

where

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}z, \quad z \sim \mathcal{N}(0, I).$$

This is the practical training objective: sample a data point, sample a time t , add noise to obtain x_t , and train a neural network to predict the noise.

10.1 Connection to the KL Term

For Gaussian distributions with the same covariance,

$$\mathcal{N}(m_1, \Sigma) \quad \text{and} \quad \mathcal{N}(m_2, \Sigma),$$

the KL divergence is

$$D_{\text{KL}}(\mathcal{N}(m_1, \Sigma) \parallel \mathcal{N}(m_2, \Sigma)) = \frac{1}{2}(m_2 - m_1)^\top \Sigma^{-1}(m_2 - m_1).$$

Therefore, the KL term

$$D_{\text{KL}}(q(x_{t-1} \mid x_t, x_0) \parallel p_{\theta}(x_{t-1} \mid x_t))$$

becomes, up to constants and weights, a squared error between the true posterior mean and the predicted mean.

Since the predicted mean is parametrized through ε_{θ} , this becomes a weighted squared error between the true noise z and the predicted noise $\varepsilon_{\theta}(x_t, t)$. The weighted objective can be written in the form

$$\frac{1}{2} \frac{\beta_t}{\sigma_t^2 \bar{\alpha}_t} \mathbb{E} \left[\|z - \varepsilon_{\theta,t}(x_t)\|^2 \right]$$

up to constants, and the simplified unweighted form is used in practice.

11 Score Function and Langevin Motivation

Diffusion models are also connected with score-based generative modeling.

Let

$$X \sim q(x).$$

The score function is

$$s(x) = \nabla_x \log q(x).$$

If we know the score, we can sample from q using Langevin dynamics.

11.1 Unadjusted Langevin Algorithm

The Unadjusted Langevin Algorithm is

$$X_{k+1} = X_k + \gamma \nabla_x \log q(X_k) + \sqrt{2\gamma} \epsilon_k, \quad \epsilon_k \sim \mathcal{N}(0, I).$$

Under suitable conditions and as $k \rightarrow \infty$ with appropriate step size behavior, the distribution of X_k approaches q .

The key idea is:

$$\text{to sample from } q(x), \text{ it is enough to know } \nabla_x \log q(x).$$

In diffusion models, the neural network trained to denoise can be interpreted as learning information related to the score of the noisy distributions.

12 Connection Between Noise Prediction and Score Matching

For the forward marginal

$$q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, \sigma_t^2 I),$$

we have

$$\nabla_{x_t} \log q(x_t | x_0) = -\frac{1}{\sigma_t^2} (x_t - \sqrt{\bar{\alpha}_t} x_0).$$

Using

$$x_t - \sqrt{\bar{\alpha}_t} x_0 = \sigma_t z,$$

we obtain

$$\nabla_{x_t} \log q(x_t | x_0) = -\frac{z}{\sigma_t}.$$

Therefore, predicting z is equivalent to predicting the conditional score up to a factor:

$$z = -\sigma_t \nabla_{x_t} \log q(x_t | x_0).$$

The unconditional score of the noisy marginal $q_t(x_t)$ is

$$\nabla_{x_t} \log q_t(x_t).$$

By conditioning on x_0 , one can show that

$$\nabla_{x_t} \log q_t(x_t) = \mathbb{E}[\nabla_{x_t} \log q(x_t | x_0) | x_t].$$

Thus, denoising score matching and noise prediction are closely related.

13 Training Algorithm

Algorithm 1 Training a Denoising Diffusion Model

Require: Dataset $\mathcal{D}$, noise schedule $\{\beta_t\}_{t=1}^T$, neural network ε_θ , learning rate η .

- 1: Define $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$.
- 2: For training iterations do:

3: Sample $x_0 \sim \mathcal{D}$.

4: Sample $t \sim \text{Uniform}\{1, \dots, T\}$.

5: Sample $z \sim \mathcal{N}(0, I)$.

6: Form the noisy sample

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}z.$$

7: Predict the noise $\varepsilon_\theta(x_t, t)$.

8: Compute

$$L(\theta) = \|z - \varepsilon_\theta(x_t, t)\|^2.$$

9: Update

$$\theta \leftarrow \theta - \eta \nabla_\theta L(\theta).$$

14 Sampling Algorithm

After training, sampling starts from Gaussian noise:

$$x_T \sim \mathcal{N}(0, I).$$

Then one applies the learned reverse transitions for $t = T, T-1, \dots, 1$.

A common DDPM sampling step is

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_\theta(x_t, t) \right) + \sigma_t z,$$

where $z \sim \mathcal{N}(0, I)$ for $t > 1$ and $z = 0$ for $t = 1$.

Algorithm 2 Sampling from a Trained Diffusion Model

Require: Trained network ε_θ , noise schedule $\{\beta_t\}_{t=1}^T$.

1: Sample $x_T \sim \mathcal{N}(0, I)$.

2: For $t = T, T-1, \dots, 1$ do:

3: If $t > 1$, sample $z \sim \mathcal{N}(0, I)$; otherwise set $z = 0$.

4: Compute

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_\theta(x_t, t) \right) + \sigma_t z.$$

Return x_0 .

15 Summary

Diffusion models are built around the following chain of ideas:

1. Define a forward Markov chain that gradually corrupts data:

$$q(x_t | x_{t-1}) = \mathcal{N}(\sqrt{\alpha_t}x_{t-1}, \beta_t I).$$

2. Derive the closed form:

$$q(x_t \mid x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I).$$

3. Learn a reverse Markov chain:

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(\mu_\theta(x_t, t), \Sigma_t).$$

4. Use Gaussian conditioning to derive the true posterior reverse transition:

$$q(x_{t-1} \mid x_t, x_0).$$

5. Parametrize the reverse mean through a neural network predicting the noise:

$$\varepsilon_\theta(x_t, t) \approx z.$$

6. Train by minimizing the denoising loss:

$$\mathbb{E} \left[\|z - \varepsilon_\theta(x_t, t)\|^2 \right].$$

7. Generate by starting from $x_T \sim \mathcal{N}(0, I)$ and applying the learned reverse chain.

The most important sentence is:

A diffusion model learns to reverse a gradual Gaussian noising process.